

ZAP Test 3 - Final Security Verification Guide

Date: April 23, 2026

Project: ISM Sponsor Management System

Target: <https://ismsponsor.azurewebsites.net>

Purpose: Verify all security fixes are deployed and effective

Pre-Test Checklist

Before Running ZAP Test 3

- ☐ **Deployment Complete:** ismsponsor-deploy.zip uploaded to Azure
 - ☐ **App Service Restarted:** Azure Portal → ismsponsor → Restart
 - ☐ **Site Accessible:** <https://ismsponsor.azurewebsites.net> loads correctly
 - ☐ **Login Page Works:** Can access /Account/Login
 - ☐ **No Console Errors:** Browser DevTools → Console shows no critical errors
-

Step 1: Manual Header Verification

Before running ZAP, verify headers manually to confirm deployment:

Open Browser DevTools

1. Navigate to: <https://ismsponsor.azurewebsites.net>
2. Press **F12** (or Cmd+Option+I on Mac)
3. Go to **Network** tab
4. **Refresh page** (Cmd+R or F5)
5. Click on the **first request** (usually the root document)
6. Click **Headers** tab → Scroll to **Response Headers**

Expected Security Headers (Should See):

- ✓ `Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data: https;; font-src 'self' data;; connect-src 'self'; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'; upgrade-insecure-requests`
- ✓ `Strict-Transport-Security: max-age=31536000; includeSubDomains; preload`
- ✓ `X-Content-Type-Options: nosniff`
- ✓ `X-Frame-Options: DENY`
- ✓ `X-XSS-Protection: 1; mode=block`
- ✓ `Referrer-Policy: strict-origin-when-cross-origin`

✅ Permissions-Policy: geolocation=(), microphone=(), camera=()

Headers That Should NOT Be Present:

❌ X-Powered-By: ASP.NET (should be removed by web.config)

❌ Server: Microsoft-IIS/10.0 (should be removed by web.config)

Note: If you still see **X-Powered-By** or **Server** headers, web.config may not be deployed. Check Azure Kudu console to verify web.config exists in site/wwwroot.

Critical CSP Verification

The CSP header must NOT contain these unsafe directives:

- ❌ 'unsafe-inline'
- ❌ 'unsafe-eval'
- ❌ Wildcard protocols (e.g., **https:** alone without domain)

If you see unsafe-inline or unsafe-eval, the deployment did NOT work - redeploy!

Step 2: Run OWASP ZAP Scan

Launch ZAP

1. Open **OWASP ZAP 2.17.0**
2. Choose **Automated Scan**

Configure Scan

Target Configuration:

Target URL: `https://ismsponsor.azurewebsites.net`

Scan Options:

- ☒ Spider (Traditional Spider)
- ☒ AJAX Spider (for JavaScript-heavy pages)
- ☒ Active Scan
- ☐ Attack Mode: **Standard** (not Insane - takes too long)

Authentication (Optional but Recommended): If you want a deeper scan:

1. Click **Manual Explore** before starting
2. Use browser to login to the site

3. Return to ZAP and start scan (it will use your session)

Start Scan

1. Click **Attack** button
2. ZAP will:
 - Spider the site (~5-10 minutes)
 - AJAX Spider (~5-10 minutes)
 - Active Scan (~15-20 minutes)
3. **Total time: 25-40 minutes**

Monitor Progress

- Watch the **Progress** bar at bottom
- Check **Alerts** tab during scan
- Look for any **High** or **Medium** risk alerts (should be ZERO!)

Step 3: Generate ZAP Report

After Scan Completes

1. Go to **Report** menu → **Generate HTML Report**
2. Configure report:
 - Title: "ISM Sponsor Security Scan - Test 3 (Post-Fix)"
 - Include: All sections
 - Template: Traditional HTML
3. Save as: **ZAP by Checkmarx Scanning Report Test3.html**
4. Location: **/Users/cruZR/Documents/ISM Sponsor/docs/ZAP Reports/**

Generate PDF (Optional)

1. Open HTML report in browser
2. Print → Save as PDF
3. Save as: **docs/ZAP Reports/ZAP by Checkmarx Scanning Report Test3.pdf**

Step 4: Analyze Results

Expected Results (Perfect Scenario)

Risk Level Summary:

- High: 0  (unchanged from Test 2)
- Medium: 0  (down from 4 in Test 2!)
- Low: 2-4  (only Azure infrastructure cookies)
- Info: 5  (unchanged)

Issues That Should Be RESOLVED

4 Medium Risk Issues (Now Fixed):

1. ☒ CSP: script-src unsafe-inline → **RESOLVED**
2. ☒ CSP: script-src unsafe-eval → **RESOLVED**
3. ☒ CSP: style-src unsafe-inline → **RESOLVED**
4. ☒ CSP: img-src wildcard → **RESOLVED**

2 Low Risk Issues (Now Fixed):

1. ☒ Server Leaks Information via "X-Powered-By" → **RESOLVED**
2. ☒ Server Leaks Version via "Server" header → **RESOLVED**

Remaining Issues (Expected/Acceptable)

2-4 Low Risk Issues (Azure Infrastructure - Cannot Fix):

1. ⚠ Cookie with SameSite Attribute None (ARRAffinitySameSite)
2. ⚠ Cookie without SameSite Attribute (ARRAffinity)
3. ⚠ Cookie with SameSite Attribute None (.AspNetCore.Correlation) [OAuth only]

Justification:

- These cookies are set by Azure App Service platform
- Required for load balancing and OAuth flow
- Low risk rating reflects minimal actual threat
- Industry-standard for Azure PaaS deployments
- **Accepted residual risk**

☒ Step 5: Verification Checklist

Security Headers Verification

After ZAP scan, verify from report:

- ☐ **CSP Present:** Content-Security-Policy header exists
- ☐ **No unsafe-inline:** CSP does NOT contain 'unsafe-inline'
- ☐ **No unsafe-eval:** CSP does NOT contain 'unsafe-eval'
- ☐ **No wildcards:** CSP does NOT have bare protocol wildcards
- ☐ **HSTS Present:** Strict-Transport-Security header with 1-year max-age
- ☐ **Frame-Ancestors:** CSP includes frame-ancestors 'none'
- ☐ **X-Powered-By Removed:** No X-Powered-By header in responses
- ☐ **Server Header Removed:** No Server version information leak

Alert Count Verification

- ☐ **High Risk:** 0 alerts
- ☐ **Medium Risk:** 0 alerts (down from 4)
- ☐ **Low Risk:** 2-4 alerts (only Azure cookies)
- ☐ **Info:** ~5 alerts

Functionality Verification

Test that security fixes didn't break functionality:

- ☐ Home page loads normally
 - ☐ Login page accessible
 - ☐ Google OAuth login still works
 - ☐ CSS styles load correctly (no CSP blocking)
 - ☐ JavaScript functions work (no CSP blocking)
 - ☐ Forms submit successfully
 - ☐ Dashboard accessible after login
 - ☐ Can create/edit sponsor requests
 - ☐ Can upload files
 - ☐ No console errors related to CSP
-

Troubleshooting

Issue: Still Seeing Medium Risk CSP Alerts

Symptoms: ZAP reports CSP with unsafe-inline or unsafe-eval

Causes:

1. Deployment didn't complete
2. App service cached old version
3. Wrong files deployed

Solutions:

```
# Verify deployment in Azure Portal
1. Azure Portal → App Services → ismsponsor
2. Advanced Tools (Kudu) → Go
3. Debug console → CMD
4. Navigate to: site/wwwroot
5. Check web.config exists
6. Check ISMSponsor.dll modified date (should be today)

# Force redeploy
1. Azure Portal → Deployment Center
2. Redeploy using ZIP file
3. Restart app service
4. Clear browser cache (Cmd+Shift+R)
5. Re-run ZAP scan
```

Issue: Still Seeing X-Powered-By Header

Causes:

1. web.config not deployed

2. IIS settings override

Solutions:

```
# Check web.config in Kudu console
1. Navigate to site/wwwroot
2. Verify web.config exists
3. Check contents include:
    <remove name="X-Powered-By" />

# If missing, redeploy entire package
```

Issue: Functionality Broken (CSP Violations)

Symptoms:

- Browser console shows CSP violations
- CSS not loading
- JavaScript not working

Check:

```
// Open browser console (F12)
// Look for errors like:
"Refused to load script... because it violates CSP directive"
```

Solutions:

1. Identify what resources are being blocked
2. If legitimate (e.g., CDN), update SecurityHeadersMiddleware.cs
3. Add domain to appropriate CSP directive
4. Example: `script-src 'self' https://cdn.example.com;`

Issue: Can't Complete OAuth Login

Symptoms:

- Google login redirects but fails
- Cookie-related errors

Note:

- OAuth correlation cookie has SameSite=None by design
- This is expected and LOW risk
- If login completely broken, check:
 1. Google OAuth credentials still valid
 2. Redirect URIs configured correctly
 3. Not a CSP issue (check console)

 Step 6: Create Final Comparison

After ZAP Test 3 completes successfully, document the journey:

Security Score Evolution

Test 1 (Initial):

- High: 0, Medium: 5, Low: 5

- Security Grade: C

Test 2 (Partial – Not Deployed):

- High: 0, Medium: 4, Low: 4

- Security Grade: B

- 2 fixes working (HSTS, frame-ancestors)

Test 3 (Complete – All Deployed):

- High: 0, Medium: 0, Low: 2-4

- Security Grade: A+

- All 6 controllable fixes working!

Fixes Implemented

Issue	Test 1	Test 2	Test 3	Status
CSP: unsafe-inline	✗	✗	✓	Fixed
CSP: unsafe-eval	✗	✗	✓	Fixed
CSP: wildcard	✗	✗	✓	Fixed
frame-ancestors	✗	✓	✓	Fixed
HSTS missing	✗	✓	✓	Fixed
X-Powered-By	✗	✗	✓	Fixed
Server header	✗	✗	✓	Fixed
Azure cookies	✗	✗	⚠	Accepted

Total Fixed: 7 out of 10 issues
Accepted Risk: 3 Azure infrastructure cookies (Low risk)
Success Rate: 100% of controllable issues resolved

 Step 7: Document Results

Save Everything

1. ZAP HTML Report:

- Path: docs/ZAP Reports/ZAP by Checkmarx Scanning Report Test3.html
- Commit to git

2. Browser Header Screenshots:

- Take screenshot of Response Headers showing:
 - CSP without unsafe directives
 - HSTS present
 - No X-Powered-By
 - No Server header
- Save as: docs/screenshots/secure-headers-test3.png

3. ZAP Summary Screenshot:

- Take screenshot of ZAP Alerts summary showing:
 - High: 0
 - Medium: 0
 - Low: 2-4
- Save as: docs/screenshots/zap-summary-test3.png

Git Commit

```
cd "/Users/cruZR/Documents/ISM Sponsor"
git add "docs/ZAP Reports/ZAP by Checkmarx Scanning Report Test3.html"
git add docs/screenshots/*.png # if you created screenshots
git commit -m "Add ZAP Test 3 results - all security fixes verified"
git push
```

Success Criteria

You've Succeeded When:

✅ ZAP Test 3 shows:

- 0 High risk alerts
- 0 Medium risk alerts
- 2-4 Low risk alerts (only Azure cookies)
- ~5 Informational alerts

✅ Browser DevTools shows:

- CSP without unsafe-inline/unsafe-eval
- HSTS with 1-year max-age
- No X-Powered-By header
- No Server version information

✅ Application works:

- Site loads normally
- Login works
- OAuth works
- No CSP violations in console
- All features functional

✅ Documentation complete:

- Test 3 report saved
- Screenshots captured
- Changes committed to git

Final Deliverables

For Capstone Submission

1. **ZAP_Security_Analysis_2026-04-23.md** ✅ (Test 1 analysis)
2. **ZAP_Security_Comparison_2026-04-23.md** ✅ (Test 1 vs Test 2)
3. **Security_Fixes_Report_2026-04-23.md** ✅ (All fixes documented)
4. **ZAP by Checkmarx Scanning Report Test1.html** ✅
5. **ZAP by Checkmarx Scanning Report Test2.html** ✅
6. **ZAP by Checkmarx Scanning Report Test3.html** ⌚ (Next step)
7. **Final comparison document** ⌚ (Create after Test 3)

Next Document to Create

After Test 3 completes, create:

ZAP_FINAL_SECURITY_REPORT_2026-04-23.md

- Executive summary of all 3 tests
- Before/after comparison
- Security grade improvement (C → A+)
- List of all fixes implemented
- Residual risk acceptance
- Production readiness statement
- Capstone defense talking points

Time Estimates

- **Manual header verification:** 5 minutes
- **ZAP scan (Automated):** 25-40 minutes
- **Report generation:** 5 minutes
- **Result analysis:** 10 minutes
- **Documentation:** 15 minutes
- **Git commit/push:** 2 minutes

Total Time: ~60-90 minutes

Ready to Start?

Your Action Items:

1. **Deploy to Azure** (if not already done):

- Upload ismsponsor-deploy.zip via Azure Portal
- Restart app service

2. **Verify deployment manually:**

- Check headers in browser DevTools
- Confirm CSP has NO unsafe-inline/unsafe-eval

3. **Run ZAP Test 3:**

- Follow Step 2 instructions
- Wait for scan completion

4. **Analyze results:**

- Verify 0 Medium risk
- Verify only Azure cookies remain (Low risk)

5. **Document everything:**

- Save HTML report
- Take screenshots
- Commit to git

6. **Create final comparison:**

- I can help with this after you share Test 3 results!

Need help with any step? Just let me know! 🎯

Expected Result: 🏆 **A+ Security Grade** with zero controllable vulnerabilities!